

Reviewer Pack — Minimal Rank of Tropicalized Weyl Group Orbits over Resoluti...

Entry 5f31edc102c7 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 05:06:04 UTC

Minimal Rank of Tropicalized Weyl Group Orbits over Resolution Proof Trees

Entry ID: 5f31edc102c7 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 05:06:04 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Combinatorics (Weyl Groups) **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

{‘stmt1’: ‘The minimal rank of the tropicalization of the orbit of a Weyl group action on an n -dimensional vector space is higher for resolution proofs than for DPLL proofs.’, ‘stmt2’: ‘For all $n \leq 40$ and resolution proof trees T , the minimal rank $\rho(T)$ of its associated tropicalized Weyl group orbit satisfies $\rho(T) \geq \kappa(n)$, where $\kappa(n) = \log_2(n!/(n/2)!^2)$.’, ‘stmt3’: ‘This inequality is strict for DPLL proofs with $n \leq 40$.’}

Rationale (proposer’s reasoning):

{‘ration1’: ‘Weyl group actions in algebraic combinatorics provide a rich structure that can capture the complexity of proof systems, potentially revealing new insights into resolution versus DPLL.’, ‘ration2’: ‘The use of tropicalization as a tool from algebraic geometry has been shown to be powerful in analyzing computational problems and could lead to more refined lower bounds for proof complexity.’, ‘ration3’: ‘The proposed conjecture builds on the idea that the structure of Weyl group orbits can encode information about the inherent difficulty of a proof system.’}

Taxonomy category: GCT_DET_PERM (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 0f61254ad323f4dd

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all $n \leq 40$, the mean minimal rank $\rho(T)$ of tropicalized Weyl group orbits over resolution proof trees exceeds the mean minimal rank of tropicalized Weyl group orbits over DPLL proof trees by a margin of at least $\kappa(n)$ with a 95% confidence interval. The conjecture is falsified if this margin is less than $\kappa(n)$ or any seed produces a metric value greater than 10.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "tropicalization Weyl group orbits resolution proof trees" - "minimal rank complexity theory tropicalized Weyl groups" - "Weyl group orbit resolution proof complexity analysis"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from collections import defaultdict

# Helper functions for matrix operations
def matrix_multiply(A, B):
    if len(A[0]) != len(B):
        raise ValueError("Incompatible dimensions for matrix multiplication")
    result = [[sum(a * b for a, b in zip(row_a, col_b)) for col_b in zip(*B)] for row_a in A]
    return result

def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
    augmented_matrix = [row + [i == j else 0 for j in range(cols)] for i, row in enumerate(matrix)]
    for i in range(rows):
        max_row = max(range(i, rows), key=lambda r: abs(augmented_matrix[r][i]))
        augmented_matrix[i], augmented_matrix[max_row] = augmented_matrix[max_row], augmented_matrix[i]
        if augmented_matrix[i][i] == 0:
            raise ValueError("Matrix is singular")
        for j in range(rows):
            if i != j:
                factor = augmented_matrix[j][i] / augmented_matrix[i][i]
                augmented_matrix[j] = [augmented_matrix[j][k] - factor * augmented_matrix[i][k] for k in range(cols)]
    for i in range(rows):
        augmented_matrix[i][-1] /= augmented_matrix[i][i]
        augmented_matrix[i][i] = 1
    return [row[:-1] for row in augmented_matrix]

def rank(matrix):
    reduced_matrix = gaussian_elimination(matrix)
```

```

    return sum(1 for row in reduced_matrix if any(row))

# Function to compute tropical rank
def tropical_rank(tropical_matrix):
    unique_elements = set()
    for row in tropical_matrix:
        if not isinstance(row, list):
            unique_elements.add((row,))
        else:
            unique_elements.update(map(tuple, row))
    return max(len(set(map(sum, zip(*unique_elements)))), len(unique_elements))

# Function to generate a random Weyl group orbit
def generate_weyl_orbit(n):
    identity = [i for i in range(1, n + 1)]
    orbit = [identity]
    while True:
        next_element = [x if x != 1 else 2 for x in orbit[-1]]
        if next_element == identity:
            break
        orbit.append(next_element)
    return orbit

# Function to generate a random resolution proof
def generate_resolution_proof(n):
    clauses = [[random.randint(1, n) for _ in range(random.randint(1, n))] for _ in range(random.randint(1, n))]
    return clauses

# Function to generate a random DPLL proof
def generate_dpll_proof(n):
    clauses = [[random.randint(1, n) for _ in range(random.randint(1, n))] for _ in range(random.randint(1, n))]
    return clauses

# Main function to run one trial
def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Generate a random resolution proof and DPLL proof
    n = random.randint(5, 40)
    resolution_proof = generate_resolution_proof(n)
    dpll_proof = generate_dpll_proof(n)

    # Compute the associated Weyl group orbits
    resolution_orbit = generate_weyl_orbit(n)
    dpll_orbit = generate_weyl_orbit(n)

    # Apply tropicalization to these orbits and determine their minimal rank
    resolution_rank = tropical_rank(resolution_orbit)
    dpll_rank = tropical_rank(dpll_orbit)

    # Compare the ranks of tropicalized Weyl group orbits from resolution proofs and DPLL proofs
    metric_value = resolution_rank - dpll_rank

    return {
        "metric_name": "tropical_rank_difference",
        "metric_value": metric_value,
    }

```

```

        "instances_tested": 1,
        "conjecture_holds": metric_value > 0,
        "counterexample": ""
    }

# Main block to run trials
if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(5, 35)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed=0")
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_evidence_or_budget_exceeded n_tested=30")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_41f63bd9.py", line 123, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_41f63bd9.py", line 86, in run_trial
    resolution_rank = tropical_rank(resolution_orbit)
                      ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_41f63bd9.py", line 44, in tropical_rank
    unique_elements = set(map(tuple, row))
                        ~~~~~
TypeError: 'int' object is not iterable

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the conjecture’s support or falsification. | next: Investigate and fix the error in the test code to allow for a proper evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12388
2	preregistration	ollama_remote	glm4:latest	0	0	6416
3	novelty	ollama_remote	glm4:latest	0	0	4582
4	novelty	ollama_remote	glm4:latest	0	0	5510
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17867
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15960
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14026
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13373
9	judge	ollama_remote	glm4:latest	0	0	8427

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 98549 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/5f31edc102c7.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/5f31edc102c7.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/5f31edc102c7.tar.gz (if generated)